Case Study: How NASA Uses the Spiral Model to Build Mars Rover Software

Author: Vedika Udgir, 112415232

August 11, 2026

1 Summary

Building software for a Mars rover is extremely tough. Once the rover launches, engineers cannot physically touch or fix it. The software must handle self-driving, battery management, scientific tasks, and system recovery on its own, all while dealing with long communication delays between Earth and Mars.

This case study looks at how NASA uses Barry Boehm's **Spiral Software Development Model** to build rover software. By breaking development into repeating loops focused on fixing risks early, NASA makes sure the software is reliable before sending the rover to space.

2 Introduction and Challenges

2.1 Why Deep-Space Software is Hard

Normal software models like Waterfall fail for space missions due to three big challenges:

- **Long Signal Delays:** Signals take 3 to 22 minutes to travel between Earth and Mars. Drivers on Earth cannot steer in real time, so the rover must make its own decisions.
- **No Room for Error:** A software bug can crash a billion-dollar rover off a cliff or drain its battery permanently. Fixes cannot be done in person.
- **Harsh Environment:** Mars has rough terrain, severe dust storms that block solar panels, extreme cold, and radiation that can corrupt computer memory.

2.2 Key Software Modules

The rover's Flight Software (FSW) runs on hardened flight computers (like the RAD750) and handles five main jobs:

1. **Autonomous Navigation:** Driving and avoiding rocks.
2. **Power and Heating:** Managing battery energy and heater loops.

3. **Radio Communications:** Sending and receiving data from Earth.
4. **Science Payload:** Controlling cameras, drills, and sensors.
5. **Fault Protection:** Catching errors and switching to a safe mode.

3 The Spiral Model Framework

The Spiral Model, created by Barry Boehm in 1988, is designed around **finding and fixing risks early**. Unlike fixed linear models, every spiral cycle goes through four simple steps:

1. **Set Objectives:** Decide what feature to build and identify its operational constraints.
2. **Assess and Reduce Risks:** Find what could fail, build quick test prototypes, and run experiments.
3. **Build and Code:** Write the software module for that step.
4. **Test and Plan Next Cycle:** Test the code on simulators, review progress, and plan the next cycle.

4 Step-by-Step Software Development (The Spirals)

NASA builds the software step-by-step using five major spiral loops:

4.1 Spiral 1: Core Operating System

- **Goal:** Get the basic Real-Time Operating System (RTOS) running reliably on flight hardware.
- **Big Risk:** System freezing or slow response under heavy work.
- **Fix & Test:** Build a simple scheduler prototype to test task speed and lock-up prevention.

4.2 Spiral 2: Telecommunications and Commands

- **Goal:** Allow the rover to read incoming commands from Earth and package outgoing data.
- **Big Risk:** Corrupted radio signals causing the rover to execute dangerous commands.
- **Fix & Test:** Write error-checking algorithms and test them by deliberately injecting fake signal errors.

4.3 Spiral 3: Power, Thermal, and Scientific Tools

- **Goal:** Control battery charging, night-time heaters, and scientific cameras.
- **Big Risk:** A heater staying on by mistake and draining the battery overnight.
- **Fix & Test:** Model energy consumption in simulations and test the code in cold vacuum test chambers.

4.4 Spiral 4: Self-Driving and Vision

- **Goal:** Help the rover use cameras to see rocks, map obstacles, and choose safe driving paths.
- **Big Risk:** Image processing taking too long, or the vision system confusing shadows with safe paths.
- **Fix & Test:** Test algorithms on fast coprocessors and drive physical prototype rovers in artificial sand yards.

4.5 Spiral 5: Fault Protection and Safe Mode

- **Goal:** Automatically detect hardware/software failures and put the rover in a safe, waiting state.
- **Big Risk:** The system repeatedly rebooting in a loop during an error.
- **Fix & Test:** Force fake failures (like pulling memory chips or disconnecting sensors) to prove the recovery software works cleanly.

5 Main Risks and Mitigations

Table 1 summarizes the major risks solved across the development cycles.

Risk Area	What Could Go Wrong?	Impact	How NASA Solves It
Operating System	Code hangs due to task traffic.	Severe	Set task priorities carefully (Spiral 1).
Radiation	Radiation flips memory bits in RAM.	High	Use triple-backed memory checks (Spirals 1 & 2).
Power Loss	Software leaves heaters on all night.	Catastrophic	Add hardware safety timers (Spiral 3).
Navigation	Rover gets stuck in deep sand.	Critical	Use wheel-slip detection code (Spiral 4).

Table 1: Key Risks Addressed During Spiral Cycles

6 How the Software is Tested

Testing increases in difficulty as development progresses through three levels:

1. **Software Simulators (SIL):** Pure software environments that simulate the Mars environment and CPU.
2. **Hardware Testbeds (HIL):** Running the code on actual flight computer boards connected to simulated rover parts.
3. **Mars Yard Testing:** Running the full software on real test rovers outdoors in dirt, sand, and rock yards.

7 Pros and Cons of Using the Spiral Model

7.1 Advantages

- **Fixes Big Problems Early:** Critical issues are found long before launch day.
- **Flexible to Changes:** Easy to adjust when mission goals or hardware designs change.
- **Proven Reliability:** Extensive testing reduces the chance of space mission failures.

7.2 Drawbacks

- **Expensive:** Requires many testing labs, hardware setups, and team hours.
- **Needs Experts:** Requires experienced managers to identify risks accurately.
- **Takes Time:** Running multiple spiral passes takes years of work.

8 Process Model Comparison

Table 2 compares the Spiral Model with standard software methods.

Feature	**Spiral Model (NASA)**	**Traditional Waterfall**	**Agile / Scrum**
Main Focus	Risk Reduction	Sequential Steps	Fast Feature Delivery
Cost of Changes	Low early, higher later	Very High	Low throughout
Testing Style	Continuous Hardware Testing	Testing at the end	Rapid Software Sprints
Space Suitability	**Best Fit**	High Risk of Failure	Needs Extra Structure

Table 2: Comparison of Software Process Models

9 Conclusion

Developing software for Mars requires a process that puts safety and risk management first. The Spiral Model allows NASA to systematically discover risks, build solutions, test them thoroughly, and deliver reliable software capable of driving across another planet on its own.

References

[1] B. W. Boehm, "A spiral model of software development and enhancement," *IEEE Computer*, vol. 21, no. 5, pp. 61–72, May 1988, doi: 10.1109/2.59.

[2] M. Bajracharya, M. W. Maimone, and D. DellaGiustina, "Autonomy for Mars Rovers: Past, Present, and Future," *IEEE Computer*, vol. 41, no. 12, pp. 44–50, Dec. 2008, doi: 10.1109/MC.2008.479.

[3] G. E. Reeves and A. D. Fregoso, "Testing Mars Exploration Rover Flight Software," in *IEEE Aerospace Conference Proceedings*, Big Sky, MT, USA, 2004, pp. 1–12, doi: 10.1109/AERO.2004.1368140.

[4] National Aeronautics and Space Administration (NASA), "NASA Software Catalog," 2024. [Online]. Available: `https://software.nasa.gov`.

[5] NASA Jet Propulsion Laboratory, "F Prime (F') Flight Software Framework," GitHub, 2023. [Online]. Available: `https://nasa.github.io/fprime/`.

[6] National Aeronautics and Space Administration (NASA), *NASA Software Engineering and Software Assurance Requirements*, NASA Procedural Requirements NPR 7150.2D, Mar. 2022.